RiceGuard

Rice Leaf Disease Detection System
Complete Features Documentation

Generated: August 18, 2026
Platform: Django + scikit-learn Random Forest AI Pipeline

Table of Contents

- 1. System Overview
- 2. AI Pipeline (4 Layers)
- 3. Home Page — Upload & Scan
- 4. Scan Result Page
- 5. Scan History & Location Map
- 6. Multi-Language Support (EN / Tagalog / Ilocano)
- 7. REST API
- 8. Django Admin Panel
- 9. Site Settings (Admin)
- 10. Leaf Scan Management (Admin)
- 11. Supported Diseases & Farmer Guides
- 12. Model Performance
- 13. Training & Retraining

1. System Overview

RiceGuard is a web-based rice leaf disease detection system designed for farmers and agricultural technicians in the Philippines. Users upload a photo of a rice leaf, and the system runs a multi-layer Random Forest AI pipeline to determine health status, identify specific diseases, and provide PhilRice-aligned treatment recommendations in English, Tagalog, or Ilocano.

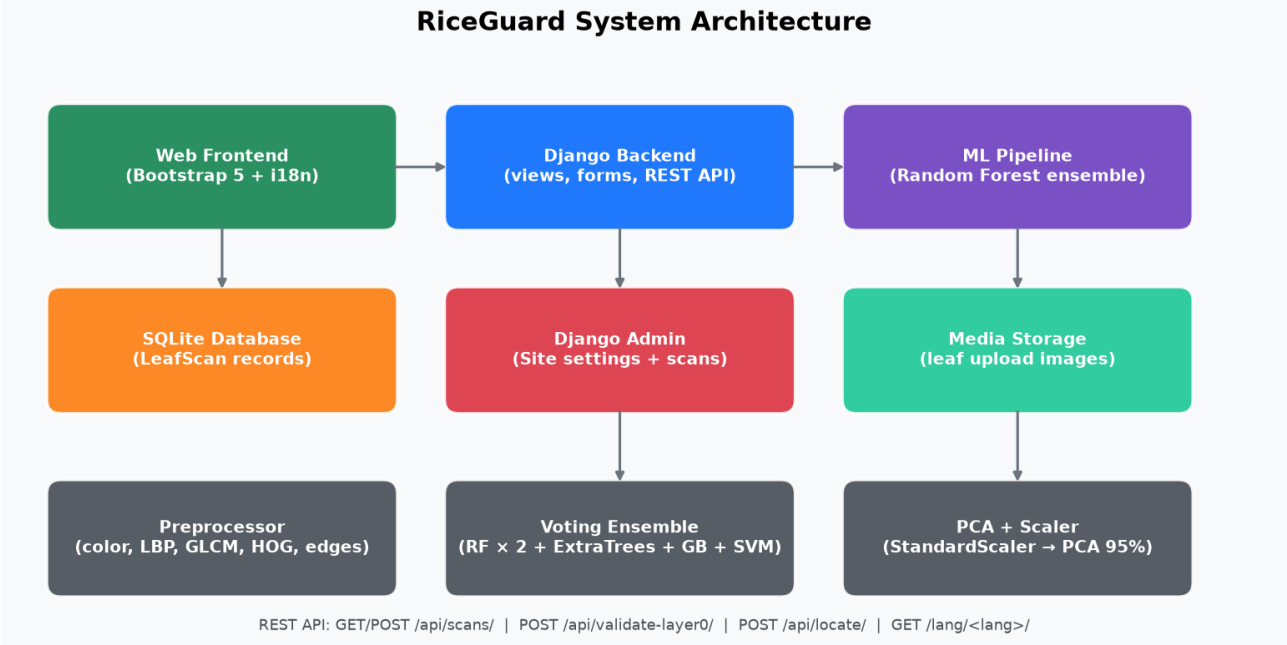


Figure 1: System architecture showing all major components.

2. AI Pipeline (4 Layers)

Every uploaded image passes through four sequential analysis layers powered by a Random Forest voting ensemble.

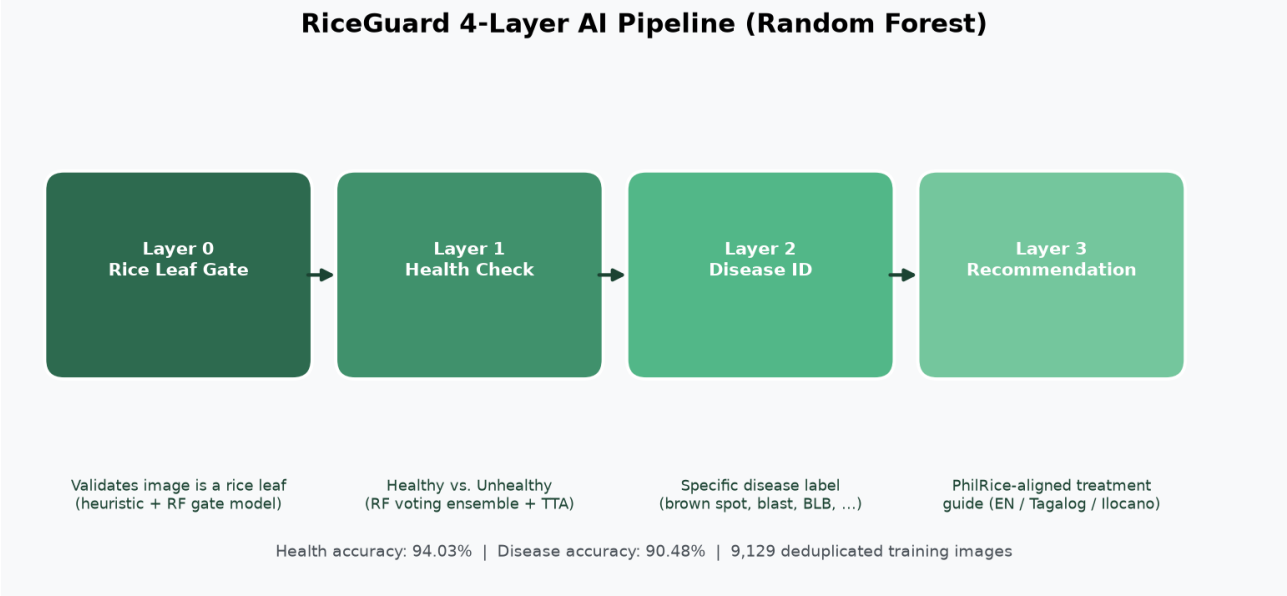


Figure 2: The 4-layer AI detection pipeline.

Layer Details

Layer 0 — Rice Leaf Gate: Validates that the uploaded image is a rice leaf using color/shape heuristics and a Random Forest gate model trained on rice vs. non-rice images. Hard rejections (not_rice / wrong_shape) stop the pipeline immediately and return an advisory result. Soft flags (low_confidence) continue analysis with a caution banner displayed on the result page.

Layer 1 — Health Check: Classifies the leaf as Healthy or Unhealthy using a soft-voting ensemble: RandomForest (weight 2) + ExtraTrees + GradientBoosting + calibrated LinearSVC, with Test-Time Augmentation (TTA) averaging probabilities over 5 augmented variants. A minimum confidence threshold prevents uncertain predictions from reaching the next layer.

Layer 2 — Disease Identification: If the leaf is unhealthy, identifies the specific disease (e.g., brown spot, blast, bacterial leaf blight) using the same voting ensemble with confidence scoring and margin checks. If the top two classes are nearly tied, the pipeline reports 'unknown' rather than guessing.

Layer 3 — Recommendation: Returns a structured farmer guide with immediate actions, full treatment procedure, cultural management, FPA-registered pesticide options, spray guidelines, and prevention advice — all based on PhilRice publications and delivered in the user's selected language (EN / Tagalog / Ilocano).

RF Ensemble Details

- Features: ~427 dimensions — color histograms, LBP texture, GLCM, HOG, and edge maps (ml/preprocessor.py).
- Ensemble: RandomForest (weight 2) + ExtraTrees + GradientBoosting + calibrated LinearSVC inside a VotingClassifier.
- StandardScaler → PCA (95% variance retained) applied before classification.

• SMOTE oversampling on the final training fit for Layers 1 and 2 (Layer 0 gate skips SMOTE).

• Hyperparameter CV: StratifiedGroupKFold (5-fold) over n_estimators × max_depth × min_samples_leaf, scored by macro-F1.

• TTA: 5 augmentation variants (original + crop/jitter/blur/flip combinations) — probabilities averaged.

• Acceptance uses max(TTA, single-pass) confidence and margin to match held-out evaluation.

3. Home Page — Upload & Scan

Key features on the home page:

- Hero banner with customizable title and tagline (editable in Admin → Site Settings).
- Drag-and-drop upload zone with file browser and camera capture support.
- Layer 0 pre-upload validation — checks the image before the full analysis runs.
- Image preview with filename, remove/change options, and warning alerts.
- Accepted formats: JPG, PNG, WEBP (max 10 MB).
- Recent scans sidebar showing the last 5 scans with health badges.
- 'How it works' section explaining the 3-step user flow.
- Language switcher in the navbar — English / Filipino (Tagalog) / Ilocano.

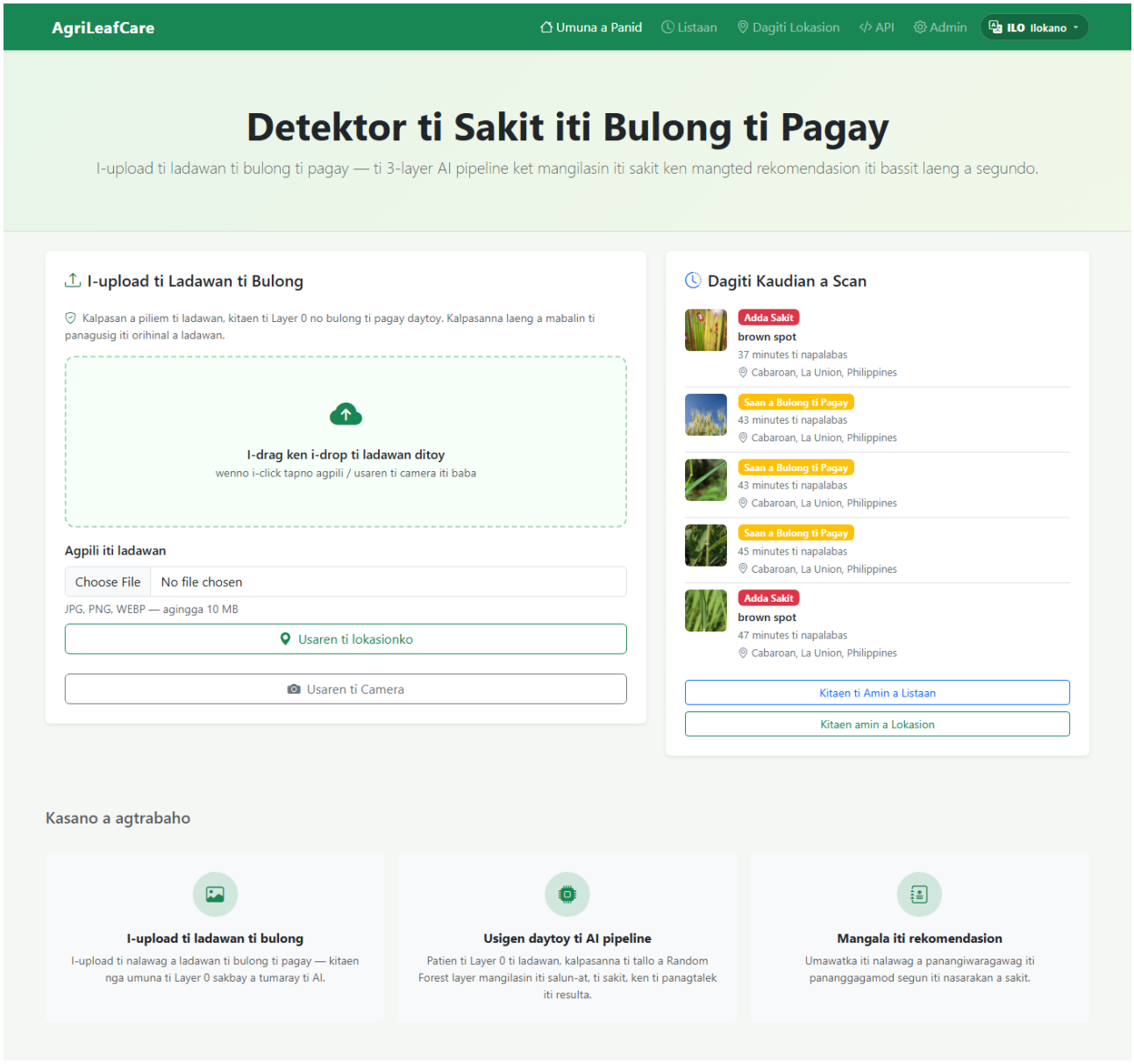


Figure 3: Home page with upload form and recent scans.

4. Scan Result Page

After analysis, users see a detailed result page with health status, disease name, confidence score, treatment recommendations, and reference training samples.

- Health status hero banner (Healthy / Unhealthy / Unknown).
- Uploaded leaf photo displayed alongside analysis confirmation.
- Summary grid: result, disease name, scan timestamp, and capture location with embedded map.
- PhilRice-aligned farmer guide with immediate actions and full treatment procedure.
- Expandable sections: symptoms, cultural management, pesticides, spray guidelines, prevention.
- Reference sample photos from the training dataset for visual comparison.
- Best training match with similarity score.
- Gate warning banner when Layer 0 had low confidence (lenient pass).
- Not-rice rejection page when Layer 0 hard-rejects the image.
- Print-friendly layout with one-click print button.
- All text translated when language cookie is set to Tagalog or Ilocano.

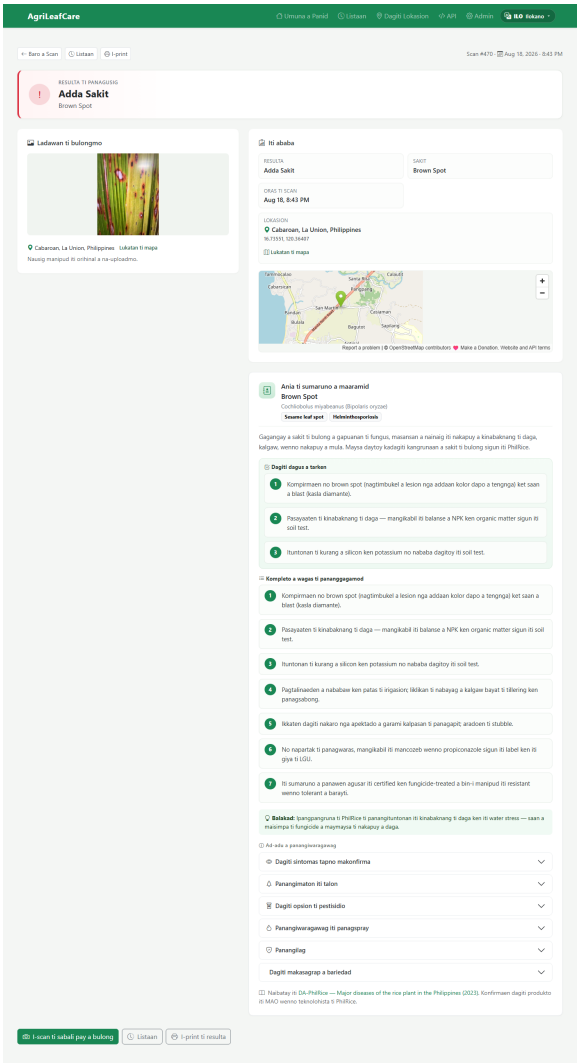


Figure 4: Disease detection result (diseased leaf example).

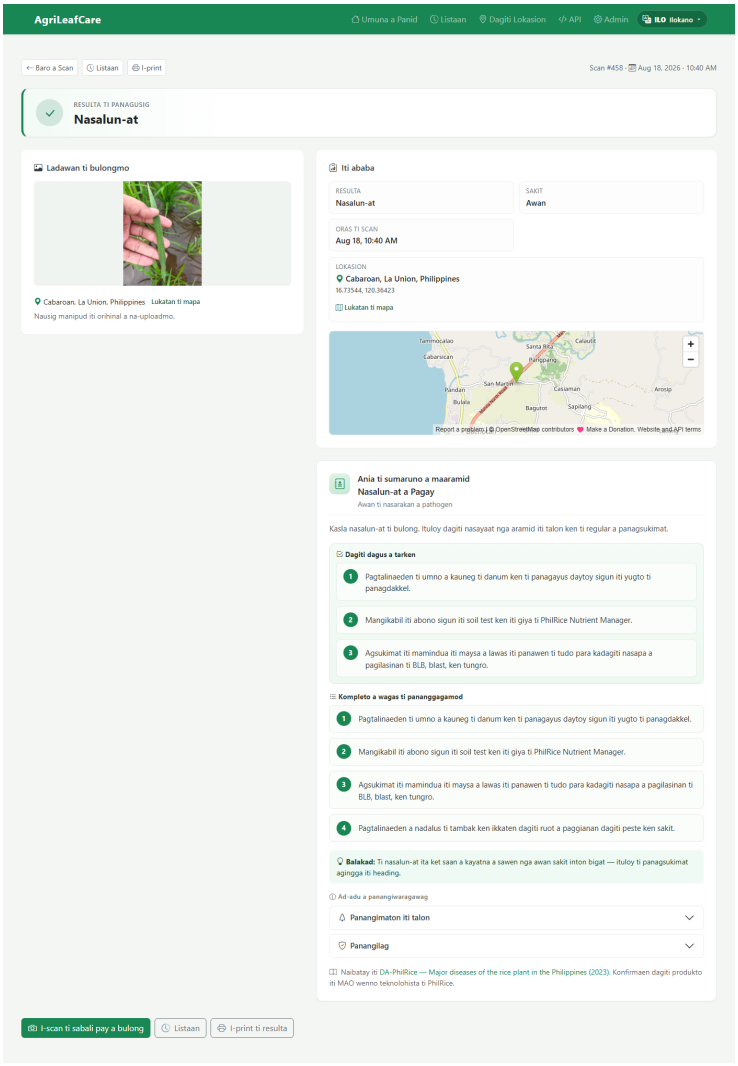


Figure 5: Healthy leaf result with maintenance guidance.

5. Scan History & Location Map

All past scans are stored and accessible from the History and Locations pages.

- Dashboard statistics: total scans, healthy count, unhealthy count.
- Filter by health status: All, Healthy, or Unhealthy.
- Card grid with thumbnail, badge, disease name, and date for each scan.
- Click any card to view the full result page.
- Scan Locations page (/locations/) — interactive OpenStreetMap showing all scan pins color-coded by health status.
- Location captured from the browser's Geolocation API at upload time (user permission required).
- Reverse-geocoded location name stored with each scan via OpenStreetMap Nominatim.
- Each scan detail page shows an embedded map when coordinates are available.

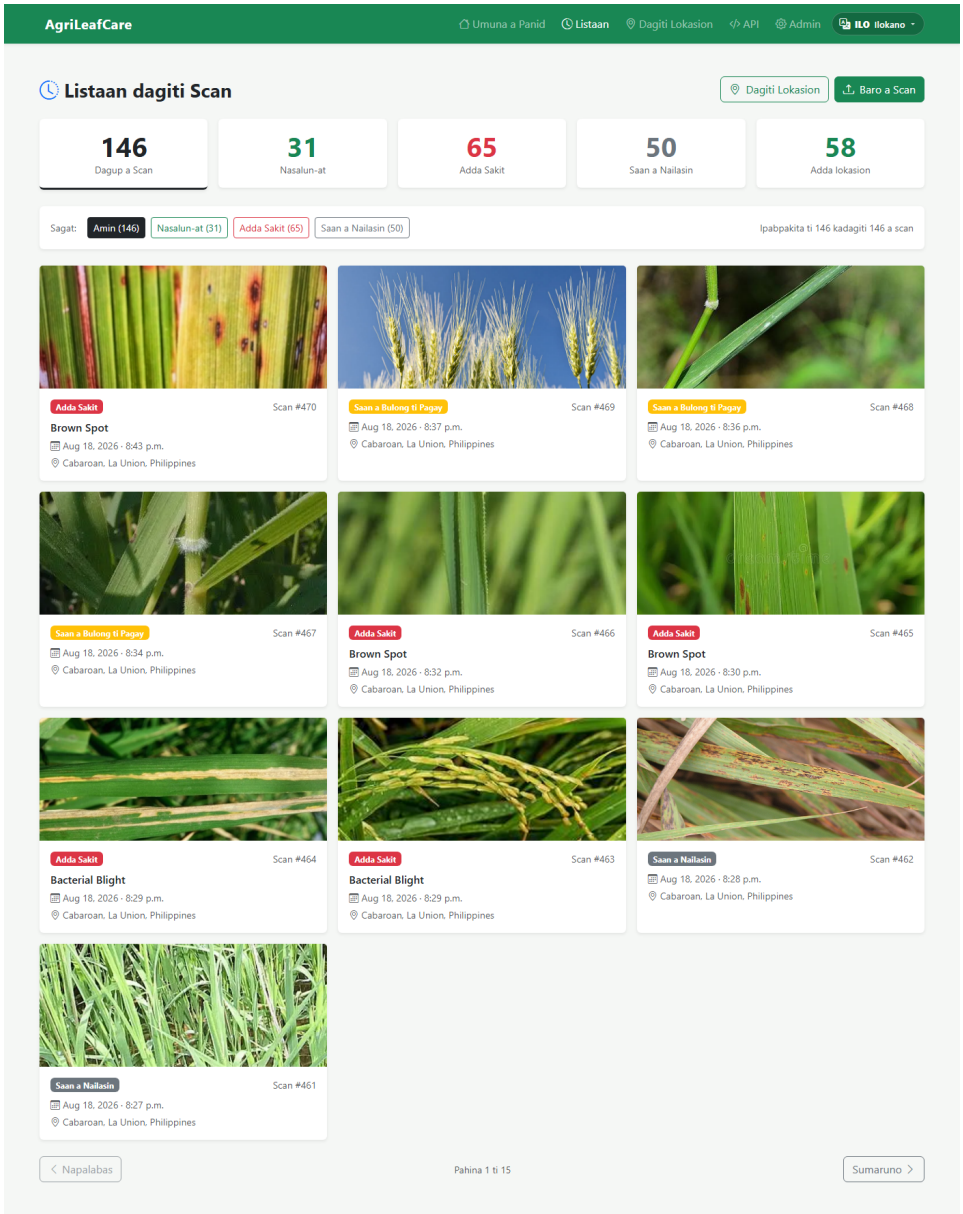


Figure 6: Scan history with statistics and filters.

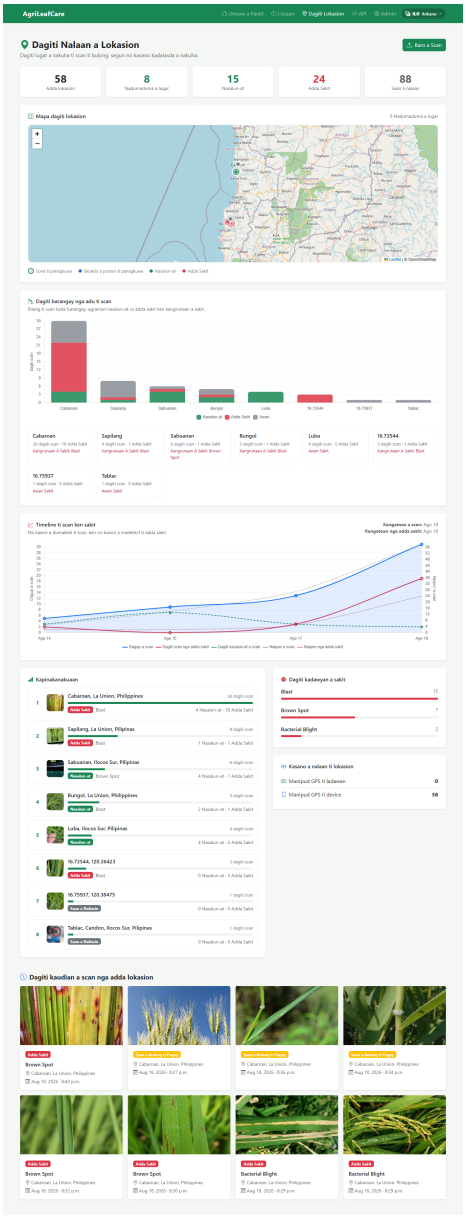


Figure 7: Scan locations map.

6. Multi-Language Support (EN / Tagalog / Ilocano)

RiceGuard supports three languages selectable at any time from the navbar language switcher. The chosen language is stored in the *riceguard_lang* browser cookie and persists across sessions.

Cookie value	Language	Notes
en	English	Default for new visitors
fil	Filipino (Tagalog)	Full UI + disease guides translated
ilo	Ilocano	Full UI + disease guides via fil_ilocano_dict glossary

- UI strings (buttons, labels, alerts, error messages) — detection/i18n.py.
- Disease guides (Tagalog) — detection/i18n_guides.py.
- Disease guides (Ilocano) — detection/i18n_guides_ilo.py.
- Pesticide section — detection/i18n_pesticides.py.
- JavaScript alert strings are injected per-language so browser-side validation messages also appear in the selected language.
- Template tag {% ui %} renders the correct string for the active language.
- Language switch URL: GET /lang// (en | fil | ilo).

7. REST API

RiceGuard exposes a REST API (Django REST Framework) for programmatic access.

Endpoint	Method	Description
/api/scans/	GET	List all scans (optional ?health= filter)
/api/scans/	POST	Upload image and run full analysis pipeline
/api/scans/<id>/	GET	Retrieve a single scan result by ID
/api/scan/	POST	Alias for POST /api/scans/
/api/validate-layer0/	POST	Pre-upload Layer 0 validation check only
/api/locate/	POST	Reverse-geocode coordinates to a location name
/lang/<lang>/	GET	Set language cookie (en | fil | ilo) and redirect

Scan API response fields: id, image_url, uploaded_at, result_health, disease_name, confidence, confidence_percent, recommendation, gate_warning, rejection_reason, processing_error, latitude, longitude, location_name, location_display, maps_url.

rejection_reason values: **not_rice** — image rejected as non-rice leaf; **wrong_shape** — wrong image shape; **low_confidence** — pipeline could not reach the confidence threshold; empty string for a normal result.

8. Django Admin Panel

Administrators access the Django Admin at /admin/ to manage site content and review all scans.

- Secure login with username and password.
- Dashboard with links to Site Settings and Leaf Scans.
- Branded header: 'Rice Leaf Disease Detector' / 'RiceGuard Admin'.

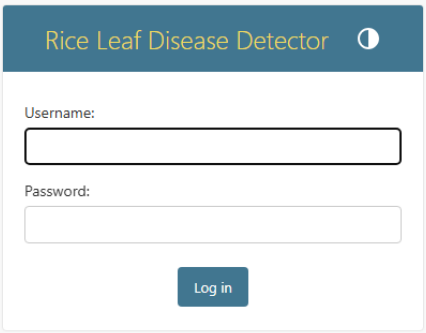


Figure 8: Admin login page.

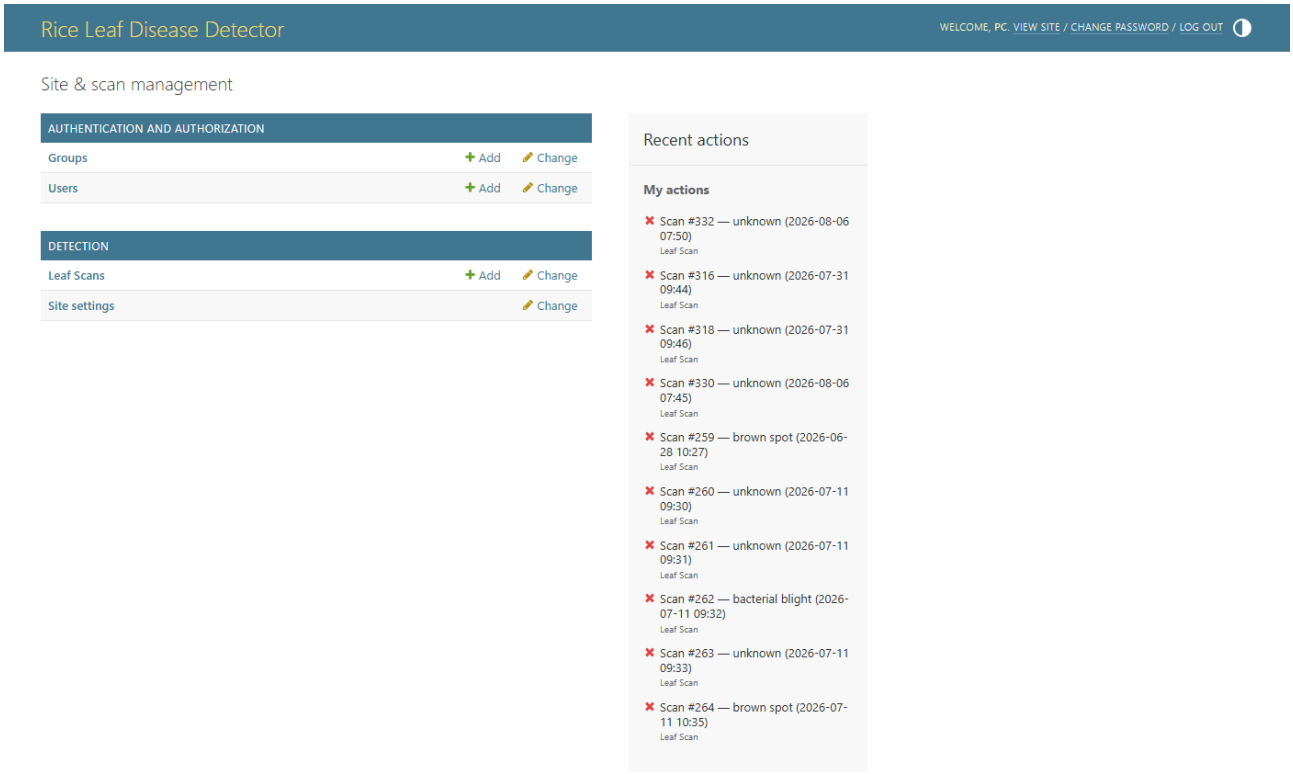


Figure 9: Admin dashboard.

9. Site Settings (Admin)

Site Settings is a singleton model — only one record exists and it controls all homepage branding.

- **Hero Title:** Main headline on the home page (default: 'Rice Leaf Disease Detector').
- **Hero Tagline:** Subtitle text shown under the headline.
- **Site Brand:** Name displayed in the navbar and browser tab (default: 'RiceGuard').
- Cannot add duplicate settings or delete the settings record.

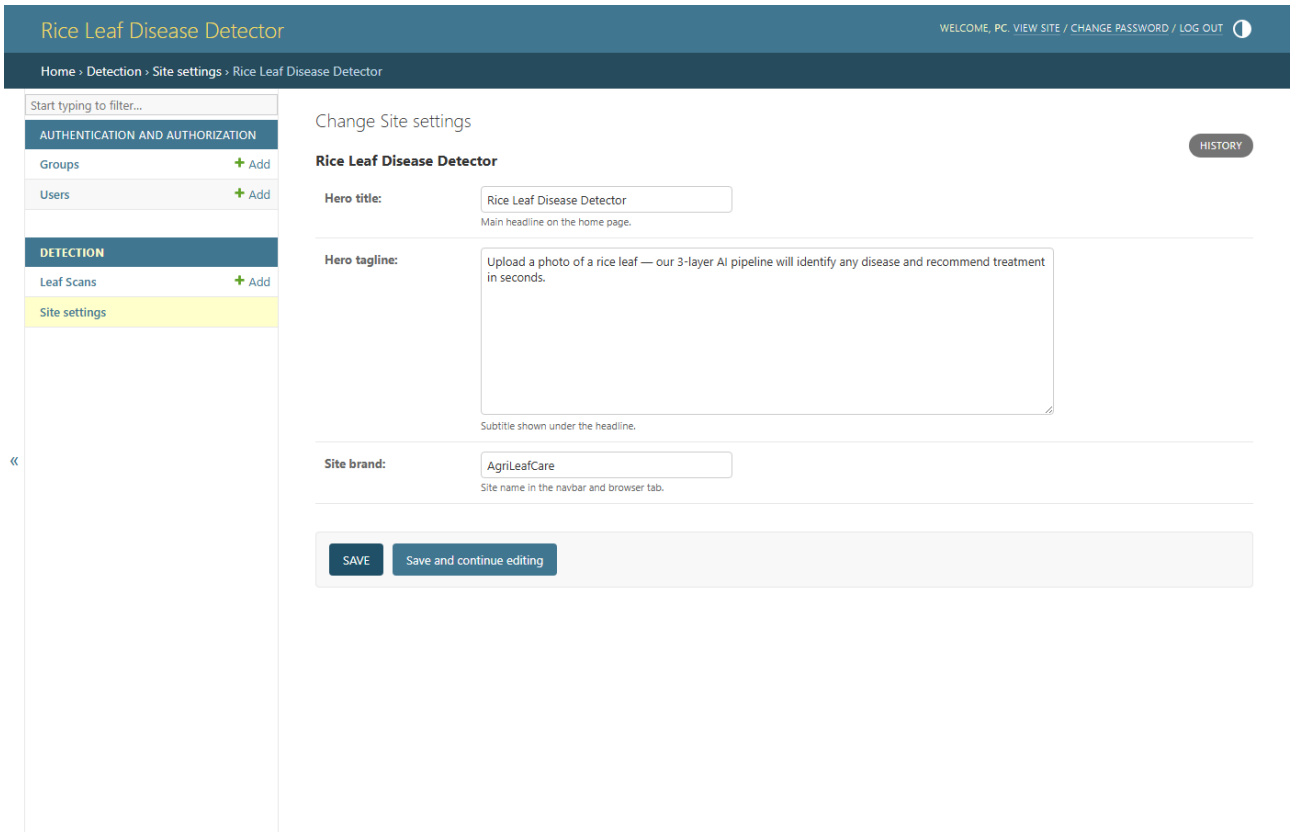


Figure 10: Site settings editor in Admin.

10. Leaf Scan Management (Admin)

Every user scan is stored as a LeafScan record and fully manageable in the Admin panel.

- List view with thumbnail, health status, disease name, confidence %, and upload date.
- Filter sidebar by health status and upload date.
- Search by disease name and recommendation text.
- Detail view with large image preview and all scan fields.
- Stored fields: image, result_health, disease_name, confidence, all_disease_probs (JSON), recommendation, processing_error, gate_warning, rejection_reason, latitude, longitude, location_name, location_source, uploaded_at.
- is_not_rice flag drives the warning badge in history and admin list views.

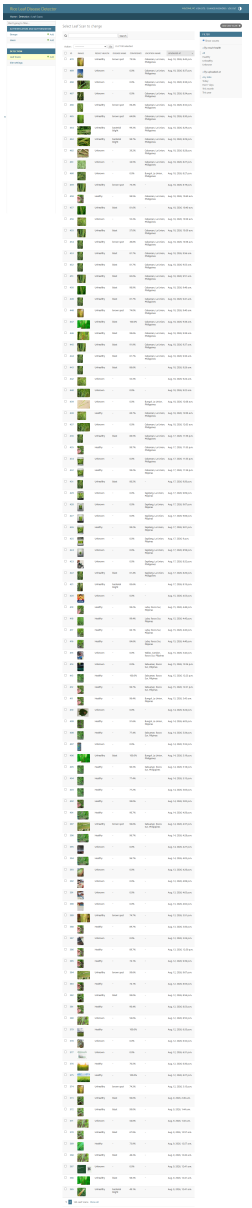


Figure 11: Leaf scan list in Admin.

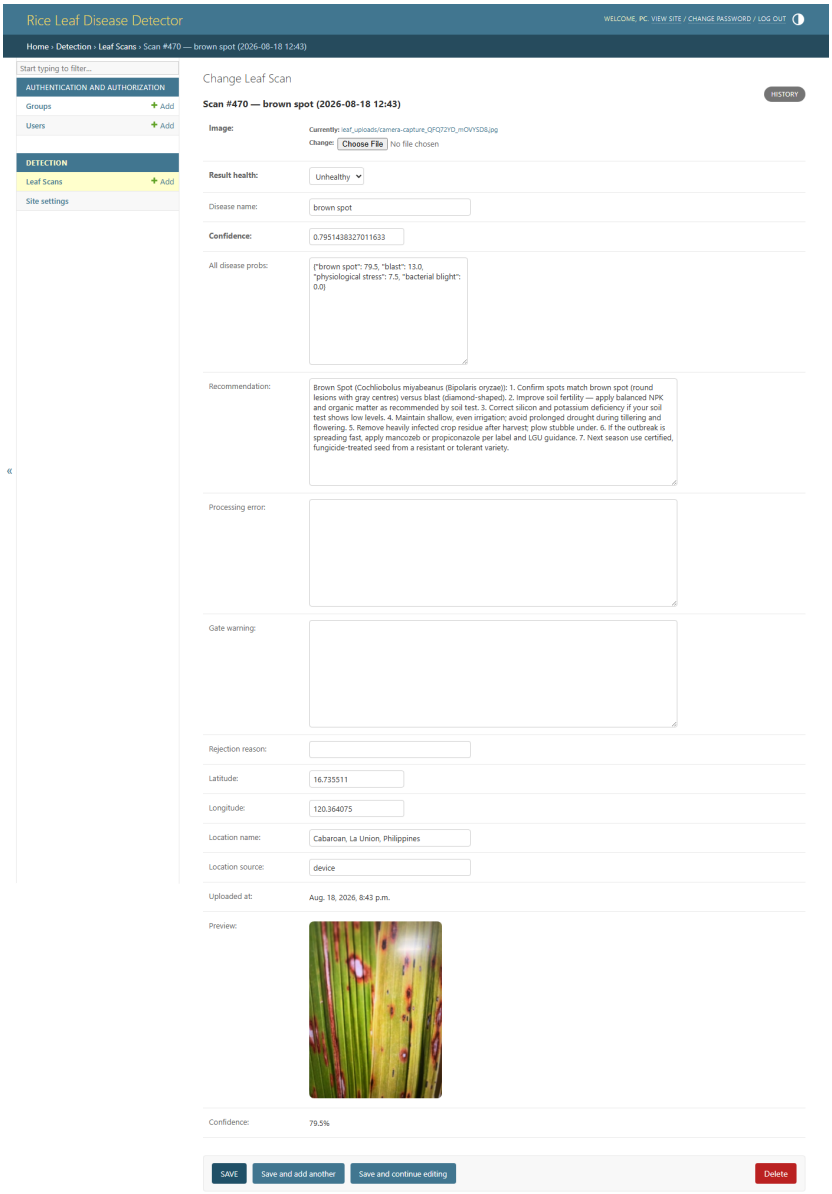


Figure 12: Individual scan detail in Admin.

11. Supported Diseases & Farmer Guides

The system detects multiple rice diseases and provides detailed PhilRice-aligned management guides in English, Filipino (Tagalog), and Ilocano.

- Bacterial Leaf Blight (BLB)
- Bacterial Leaf Streak
- Brown Spot
- False Smut
- Leaf Blast
- Leaf Scald
- Leaf Smut
- Narrow Brown Spot
- Sheath Blight
- Sheath Rot
- Tungro
- Healthy (maintenance guidance)

Each disease guide includes:

- Disease name, pathogen, and local Filipino/Ilocano names.
- Summary and symptoms to confirm diagnosis.
- Immediate actions and complete treatment procedure.
- Cultural management practices.
- FPA-registered pesticide options with rates and timing.
- Spray guidelines and prevention measures.
- Resistant rice variety recommendations.
- Farmer tips and PhilRice source citation.

12. Model Performance

Models are trained on 9,129 deduplicated images using the Random Forest voting ensemble with Test-Time Augmentation. Results on the held-out test split (20%):

Metric	Health Model	Disease Model
Accuracy	94.03%	90.48%
Precision	94.02%	91.53%
Recall	94.03%	90.48%
F1-Score	94.00%	90.98%
Test samples	1,826	1,177

Dataset: 9,129 deduplicated images (20,634 raw files; 8,035 removed — 7,528 byte-identical MD5 duplicates + 507 dHash re-encoded duplicates).
Classes: healthy, bacterial blight, blast, brown spot (train-only extras: physiological stress, senescent healthy, not-rice).
Model files: rf_health.pkl, rf_disease.pkl, rf_rice_gate.pkl
Training script: ml/train_models.py | **Evaluation:** ml/evaluate_models.py

13. Training & Retraining

All three model layers are trained from a single command. The script discovers images automatically under the data root.

Quick-start commands

```
# Full training (RF models)
python ml/train_models.py --data data/sources
```

```
# Quick smoke-test run (~600 images)
python ml/train_models.py --data data/sources --quick
```

```
# Post-train evaluation
python ml/evaluate_models.py --data data/sources
```

```
# Validate preprocessing consistency
python ml/validate_preprocessing.py --data data/sources
```

Training Pipeline Details

• Data split: 70% train / 10% validation / 20% test (stratified by class).

• 5-fold StratifiedGroupKFold cross-validation on path-level groups (prevents augmented copies leaking into the validation fold).

• Augmentation: random crop, colour jitter, blur/sharpen, Gaussian noise — scales with dataset size (more augmentation when data is scarce).

• On-the-fly augmentation applied to the final training fit only; validation and test sets remain single-pass.

• Hyperparameter CV tunes RF alone (fast) then reuses the best settings inside the full voting ensemble.

• Per-feature-block importance (color, LBP, GLCM, HOG, edges) is projected back from PCA components and printed for interpretability after each run.

• Training report saved to ml/saved_models/training_report.json; validation summary to ml/saved_models/validation_summary.json.